

C++ 프로그래밍

(템플릿 실습)

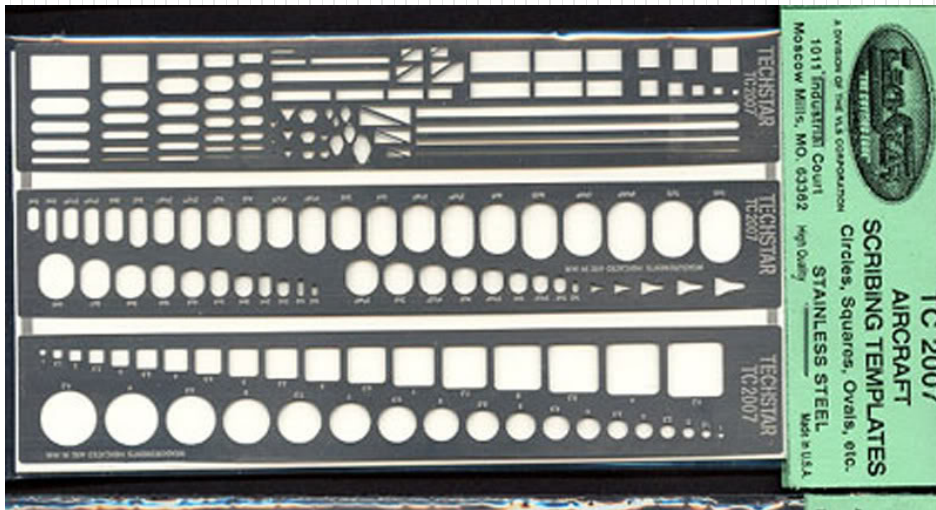
11주차

목차

- 함수 템플릿
- 클래스 템플릿
- 템플릿을 이용한 스택 구현

함수 템플릿

- 템플릿
 - 물건을 만들 때 사용되는 틀이나 모형을 의미
- 함수 템플릿
 - 함수를 찍어내기 위한 형틀



함수 템플릿 (계속)

- 일반화 프로그래밍
 - 일반적인 코드를 작성하고 이 코드를 정수나 문자열과 같은 다양한 타입의 객체에 대하여 재사용하는 프로그래밍 기법



int 버전으로 필요하시다구요.

템플릿 함수

```
___ get_max(___x , ___ y)
{
    if( x > y) return x;
    else return y;
}
```



```
int get_max(int x , int y)
{
    if( x > y) return x;
    else return y;
}
```

함수 템플릿 (계속)

- 템플릿 함수의 사용

```
template <typename T>  
T get_max(T x, T y)  
{  
    if(x > y) return x;  
    else return y;  
}
```

get_max(1, 3) 으로 호출

```
int get_max(int x, int y)  
{  
    if(x > y) return x;  
    else return y;  
}
```

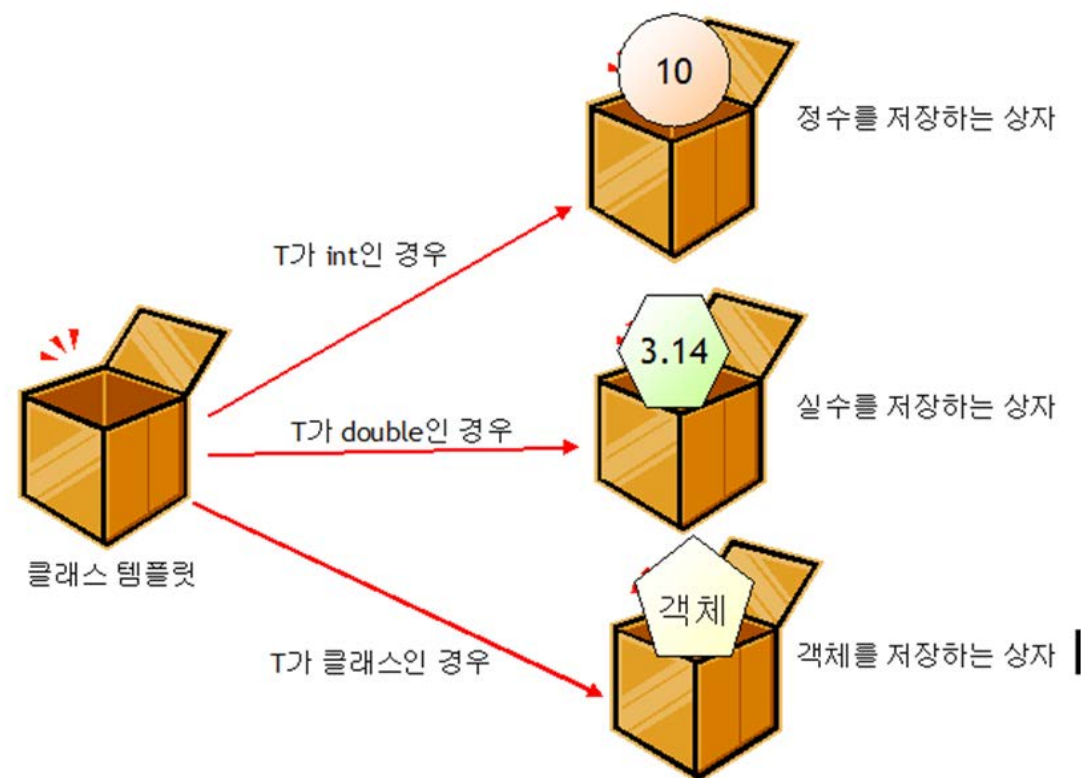
get_max(1.8, 3.7) 으로 호출

```
double get_max(double x, double y)  
{  
    if(x > y) return x;  
    else return y;  
}
```

클래스 템플릿

- 클래스 템플릿
 - 클래스를 찍어내는 틀

```
template <typename 타입이름, ...> class 클래스이름  
{  
    ...  
}
```



클래스 템플릿 (계속)

- 클래스 템플릿

```
template <typename T>
class 클래스이름
{
    ...// T를 어디서든지 사용할 수 있다.
}
```

```
#include <iostream>
using namespace std;

template <typename T>
class Box {
    T data; // T는 타입(type)을 나타낸다.
public:
    Box() { }
    void set(T value) {
        data = value;
    }
    T get() {
        return data;
    }
};
```

클래스 템플릿

```
int main()
{
    Box<int> box;
    box.set(100);
    cout << box.get() << endl;

    Box<double> box1;
    box1.set(3.141592);
    cout << box1.get() << endl;

    return 0;
}
```

정수 버전 클래스

실수 버전 클래스

실행 결과

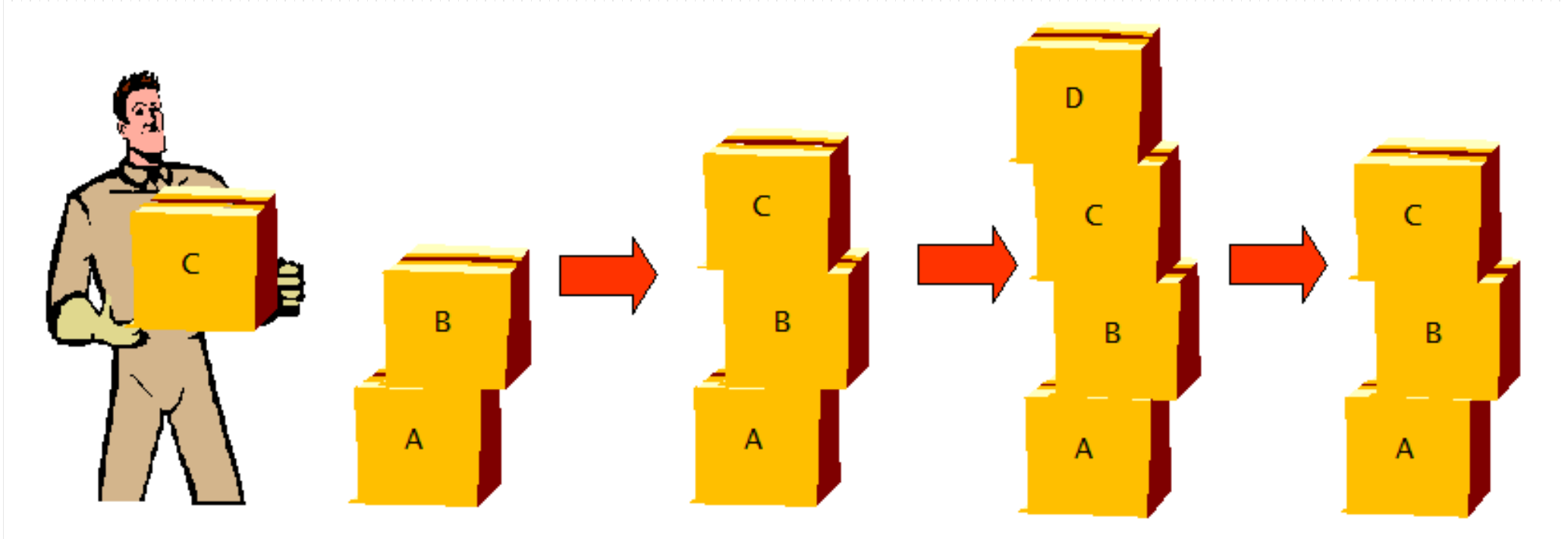
100

3.14159

계속하려면 아무 키나 누르십시오 . . .

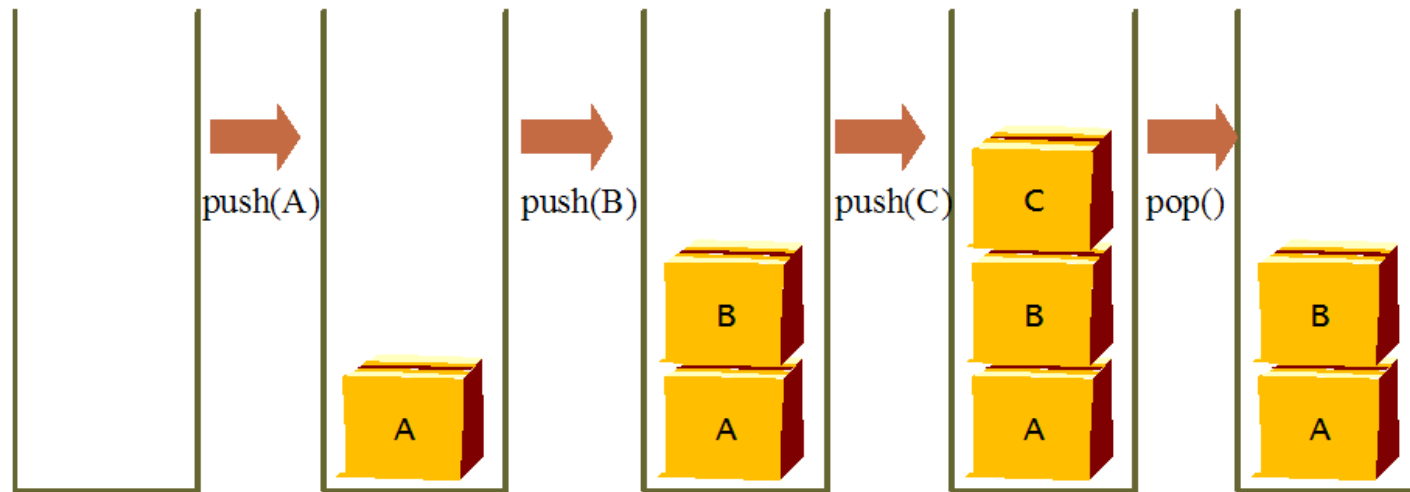
템플릿을 이용한 스택 구현

- 스택(stack)
 - 후입 선출(LIFO:Last-In First-Out)자료 구조



템플릿을 이용한 스택 구현 (계속)

- `is_empty(s) ::=` 스택이 비어있는지를 검사한다.
- `is_full(s) ::=` 스택이 가득 찼는가를 검사한다.
- `push(s, e) ::=` 스택의 맨 위에 요소 `e`를 추가한다.
- `pop(s) ::=` 스택의 맨 위에 있는 요소를 삭제한다.

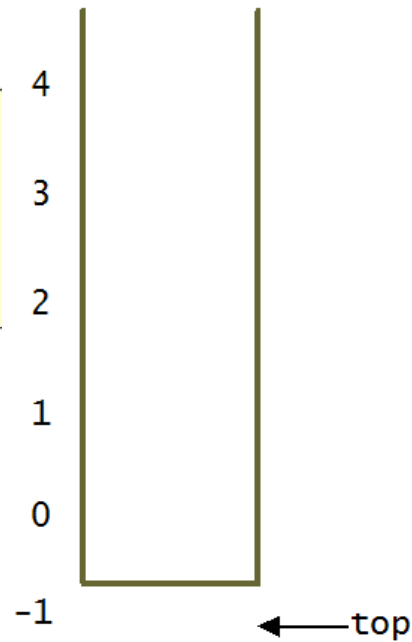


초기상태

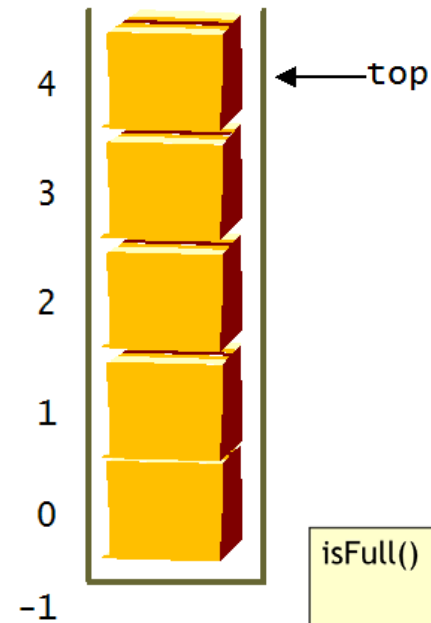
템플릿을 이용한 스택 구현 (계속)

- 스택의 공백 상태와 포화 상태

```
isEmpty()  
  
if top = -1  
  then return TRUE  
  else return FALSE
```



(a) 공백상태



(b) 포화상태

```
isFull()  
  
if top = (MAX_STACK_SIZE-1)  
  then return TRUE  
  else return FALSE
```

템플릿을 이용한 스택 구현 (계속)

- push()

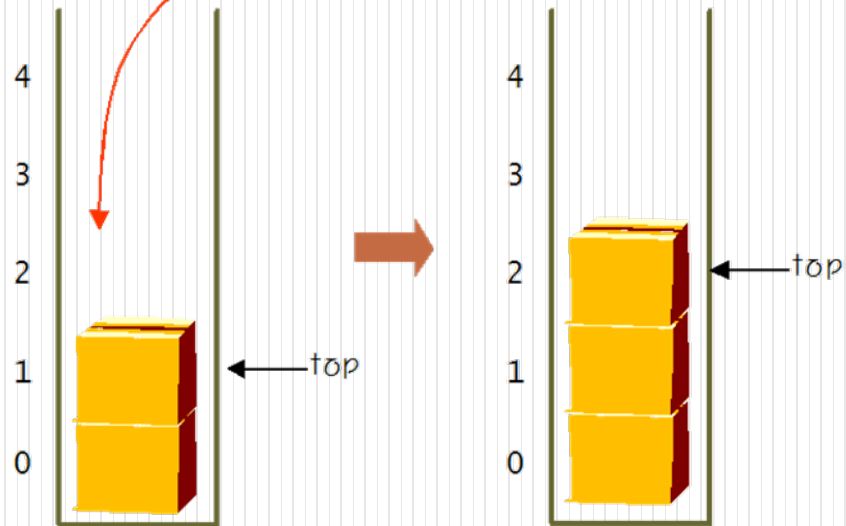
```
push(x)
```

```
if isFull()
```

```
    then error "overflow"
```

```
    else  $top \leftarrow top + 1$ 
```

```
         $stack[top] \leftarrow x$ 
```



템플릿을 이용한 스택 구현 (계속)

- pop()

```
pop(x)
```

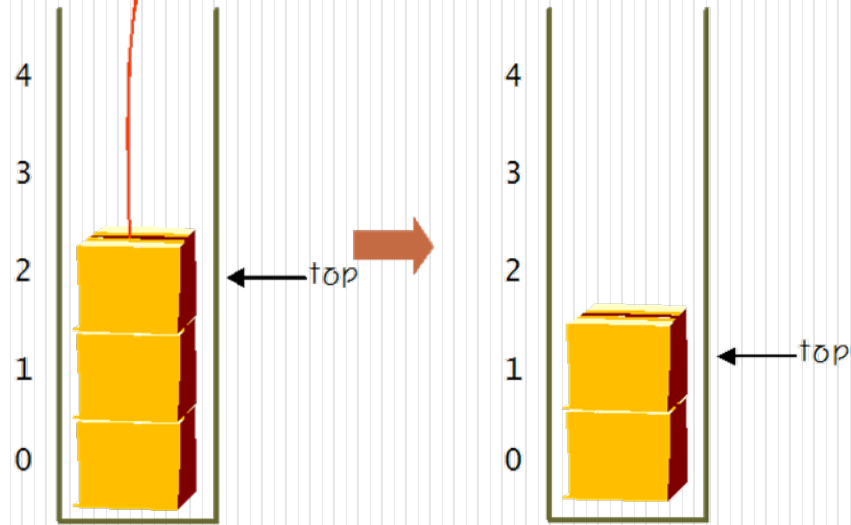
```
if isEmpty()
```

```
  then error "underflow"
```

```
  else  $e \leftarrow \text{stack}[\text{top}]$ 
```

```
        $\text{top} \leftarrow \text{top} - 1$ 
```

```
  return e
```



템플릿을 이용한 스택 구현 (계속)

- 스택의 구현

```
template <class T>
class Stack {
private:
    T* s;
    int size;
    int top;
public:
    Stack(int n = 100) : size(n), top(-1)
    {
        s = new T[size];
    }
    ~Stack() { delete []s; }
    void push(T v);
    T pop();
    bool isEmpty() const { return top == -1;}
    bool isFull() const { return top == size - 1;}
};
```

```
template< typename T >
void Stack< T >::push( T v )
{
    if ( isFull() )
    {
        cout << "stack full" << endl;
    }
    else
    {
        s[ ++top ] = v;
    }
}

template< typename T >
T Stack< T >::pop( )
{
    if ( isEmpty() )
    {
        cout << "stack empty : ";
        return -1;
    }
    else
    {
        return s[ top-- ];
    }
}
```

```
int main()
{
    Stack<int> s; // 크기가 100인 정수형 스택
    s.push(100);
    s.push(200);
    s.push(300);
    s.push(400);
    cout << s.pop() << endl;
    cout << s.pop() << endl;
    cout << s.pop() << endl;
    cout << s.pop() << endl;
    return 0;
}
```

```
400
300
200
100
계속하려면 아무 키나 누르십시오 . . .
```

HEX CODE

제어 문자			공백 문자			구두점			숫자			알파벳		
10진	16진	문자	10진	16진	문자	10진	16진	문자	10진	16진	문자	10진	16진	문자
0	0x00	NUL	32	0x20	SP	64	0x40	④	96	0x60				
1	0x01	SOH	33	0x21	!	65	0x41	A	97	0x61	a			
2	0x02	STX	34	0x22	"	66	0x42	B	98	0x62	b			
3	0x03	ETX	35	0x23	#	67	0x43	C	99	0x63	c			
4	0x04	EOT	36	0x24	\$	68	0x44	D	100	0x64	d			
5	0x05	ENQ	37	0x25	%	69	0x45	E	101	0x65	e			
6	0x06	ACK	38	0x26	&	70	0x46	F	102	0x66	f			
7	0x07	BEL	39	0x27	'	71	0x47	G	103	0x67	g			
8	0x08	BS	40	0x28	(	72	0x48	H	104	0x68	h			
9	0x09	HT	41	0x29	)	73	0x49	I	105	0x69	i			
10	0x0A	LF	42	0x2A	*	74	0x4A	J	106	0x6A	j			
11	0x0B	VT	43	0x2B	+	75	0x4B	K	107	0x6B	k			
12	0x0C	FF	44	0x2C	,	76	0x4C	L	108	0x6C	l			
13	0x0D	CR	45	0x2D	-	77	0x4D	M	109	0x6D	m			

14	0x0E	SO	46	0x2E	.	78	0x4E	N	110	0x6E	n
15	0x0F	SI	47	0x2F	/	79	0x4F	O	111	0x6F	o
16	0x10	DLE	48	0x30	0	80	0x50	P	112	0x70	p
17	0x11	DC1	49	0x31	1	81	0x51	Q	113	0x71	q
18	0x12	DC2	50	0x32	2	82	0x52	R	114	0x72	r
19	0x13	DC3	51	0x33	3	83	0x53	S	115	0x73	s
20	0x14	DC4	52	0x34	4	84	0x54	T	116	0x74	t
21	0x15	NAK	53	0x35	5	85	0x55	U	117	0x75	u
22	0x16	SYN	54	0x36	6	86	0x56	V	118	0x76	v
23	0x17	ETB	55	0x37	7	87	0x57	W	119	0x77	w
24	0x18	CAN	56	0x38	8	88	0x58	X	120	0x78	x
25	0x19	EM	57	0x39	9	89	0x59	Y	121	0x79	y
26	0x1A	SUB	58	0x3A	:	90	0x5A	Z	122	0x7A	z
27	0x1B	ESC	59	0x3B	;	91	0x5B	[	123	0x7B	{
28	0x1C	FS	60	0x3C	<	92	0x5C	\	124	0x7C	
29	0x1D	GS	61	0x3D	=	93	0x5D	]	125	0x7D	}
30	0x1E	RS	62	0x3E	>	94	0x5E	^	126	0x7E	~
31	0x1F	US	63	0x3F	?	95	0x5F	_	127	0x7F	DEL

0x00 → 1Byte

0x0001 → 2Byte

→ 0x00 0x001 → 2Byte

HEX COCE 표현

- 1Byte 표현
 - 0x00
- 2Byte 표현
 - 0x0001
 - 0x00 0x01
- 3Byte 표현
 - 0x000102
 - 0x00 0x01 0x02